Loan Default Model Report

Oct 12, 2026 · @Snehith

The tuned XGBoost model reaches 80% accuracy but finds only 8% of the loans that default. That accuracy equals the 80.0% you get by predicting "no default" for every loan, so the model needs changing before it is useful.

Objective and data

The model predicts `default` (1 = the loan defaulted, 0 = it did not) for each loan in a Kaggle loan dataset with more than 2 million raw records and over 20 features. Defaults are the minority class: 53,871 of the 269,614 test loans (20%).

Split	Share	Rows	Used for
Train	80%	about 1,078,000	Fitting and the 3-fold search
Test	20%	269,614	One final evaluation

The test size implies about 1.35 million rows were modelled, so if the raw file is larger, rows were removed during cleaning. Three columns were dropped before training: `default` (the target), `loan_status` (it encodes the outcome) and `issue_d` (the issue date). Dropping outcome-related columns stops the model from learning the answer directly.

Procedure

1. **Separate features and target.** `X` is every column except `loan_status`, `default` and `issue_d`; `Y` is `default`.
2. **Split the data.** A stratified 80/20 train/test split (`random_state=2`) keeps the 20% default rate in both sets.
3. **Build one pipeline.** `StandardScaler` feeds an `XGBClassifier` with `tree_method="hist"` (fast on large tables) and `eval_metric="logloss"`.
4. **Search hyperparameters.** `RandomizedSearchCV` tries 10 random combinations, each scored with 3-fold stratified cross-validation (30 fits) on ROC-AUC.
5. **Refit and predict.** The best combination is refit on the full training set and predicts the test set at XGBoost's default 0.5 cutoff.
6. **Evaluate.** Classification report, confusion matrix, accuracy and AUC are computed on the 20% test set only.

Hyperparameter	Values searched
`max_depth`	3, 4, 5
`learning_rate`	0.01, 0.05, 0.1
`n_estimators`	100, 200, 300
`subsample`	0.7, 0.8, 1.0
`colsample_bytree`	0.7, 0.8, 1.0

Open: the run's "Best Params" and AUC lines were not included, so they are missing here. Paste them in to complete this section.

Why XGBoost

XGBoost was chosen because gradient-boosted trees are the strongest standard choice for large tables of mixed numeric features like loan records, and they train fast with little preprocessing.

- **Proven on tabular data.** Benchmarks find tree-based models still outperform deep learning on typical tabular data (Grinsztajn et al., NeurIPS 2022).
- **Little preprocessing.** It handles skewed features such as income and loan amount, and missing values, without scaling or imputation.
- **Finds interactions automatically.** For example, credit score can matter more at certain income levels, and trees learn those splits without hand-built features.
- **Scales to this size.** The `hist` method trains on a million or more rows in minutes.
- **Explainable.** Feature importance and SHAP values show what drove each prediction, which matters in credit decisions.
- **A fair baseline.** Any later neural network has a strong reference score to beat.

Alternative	Why it was not the first choice
Logistic regression	Linear, so it misses interactions; useful only as a sanity baseline
Random forest	Usually scores below boosting, which corrects its own errors in sequence
Neural network	Needs more preprocessing and tuning; rarely beats boosting alone on tabular data; kept as a second model for blending

Alternative	Why it was not the first choice
Transformer (FT-Transformer)	Slow to train and shows no consistent edge over trees at this scale
LightGBM / CatBoost	Close competitors; better used as extra ensemble members

These alternatives were not benchmarked in this run. A same-fold comparison is listed under Further scope.

Baseline results

The model identifies non-defaulters well (recall 0.98) but catches only 8% of defaulters. These are the test-set scores after the search (3 folds × 10 candidates = 30 fits):

Class	Precision	Recall	F1	Support
0 (no default)	0.81	0.98	0.89	215,743
1 (default)	0.57	0.08	0.14	53,871
Accuracy			0.80	269,614
Macro average	0.69	0.53	0.52	269,614
Weighted average	0.76	0.80	0.74	269,614

Recall of 0.08 on 53,871 defaulters means roughly 4,300 were caught and about 49,600 were missed. Those misses are the false negatives.

The false-negative problem

The model misses most defaulters because it was never told that they matter more, and it was asked to decide at a cutoff that suits the majority class. Four causes stack up:

- **Accuracy hides it.** Predicting "no default" for every loan scores 215,743 / 269,614 = 80.0%, the same as the model's 0.80.
- **Class imbalance.** Non-defaulters outnumber defaulters 4 to 1, so an unweighted loss gains most by getting the majority right.
- **The 0.5 cutoff.** `predict()` labels a loan as default only when its probability exceeds 0.5. With a 20% base rate, few defaulters ever reach that. The search scored ROC-AUC, which ignores the cutoff, so nothing corrected it.

- **A small model for a large dataset.** Depth 3–5 with at most 300 trees is likely to underfit more than a million rows.

In lending the two errors cost different amounts: approving a loan that defaults usually costs more than rejecting a good borrower. A model that misses 92% of defaults is not usable as a risk screen.

Improvements

The biggest gain comes from choosing the decision cutoff on a separate validation set. The other changes make the model itself stronger.

Change	What to do	Effect
Validation split	Hold out 10% of the training data; use it only for early stopping and the cutoff	Keeps the test set honest
Tune the cutoff	Use `predict_proba` and pick the threshold that minimises cost (for example a missed default costs 3× a false alarm)	Raises recall directly
Class weight	Set `scale_pos_weight` = 215,743 / 53,871 ≈ 4, or compare with and without it	Makes defaults count more in training
Search metric	Score with `average_precision` instead of `roc_auc`	Prefers models that rank defaults well
Bigger model	Depth 6–10, up to 3,000 trees at learning rate 0.05, with `early_stopping_rounds=50`	Reduces underfitting

Further scope

- **Feature engineering.** Add ratios such as loan amount to income, interest rate against grade, and credit history length. This usually helps more than extra tuning.
- **Leakage audit.** Remove any column only known after the loan outcome (payments received, recoveries, outstanding principal). Lending-style data often contains them.
- **Time-based validation.** If `issue_d` is available, train on older loans and test on newer ones, which matches real use better than a random split.
- **Model comparison.** Run logistic regression, random forest, XGBoost, LightGBM and CatBoost on identical folds and report mean ± standard deviation.

- **Neural network and ensembling.** Train a tabular network with categorical embeddings, then blend its out-of-fold predictions with the tree models; a weaker network often still improves the blend.
- **Calibration.** Check calibration curves and apply isotonic or Platt calibration so predicted probabilities can be trusted.
- **Explainability and fairness.** Add SHAP summaries and check error rates across applicant groups before any real-world use.
- **Metrics to report.** ROC-AUC, PR-AUC, recall at a fixed precision, and the expected cost of errors, rather than accuracy.